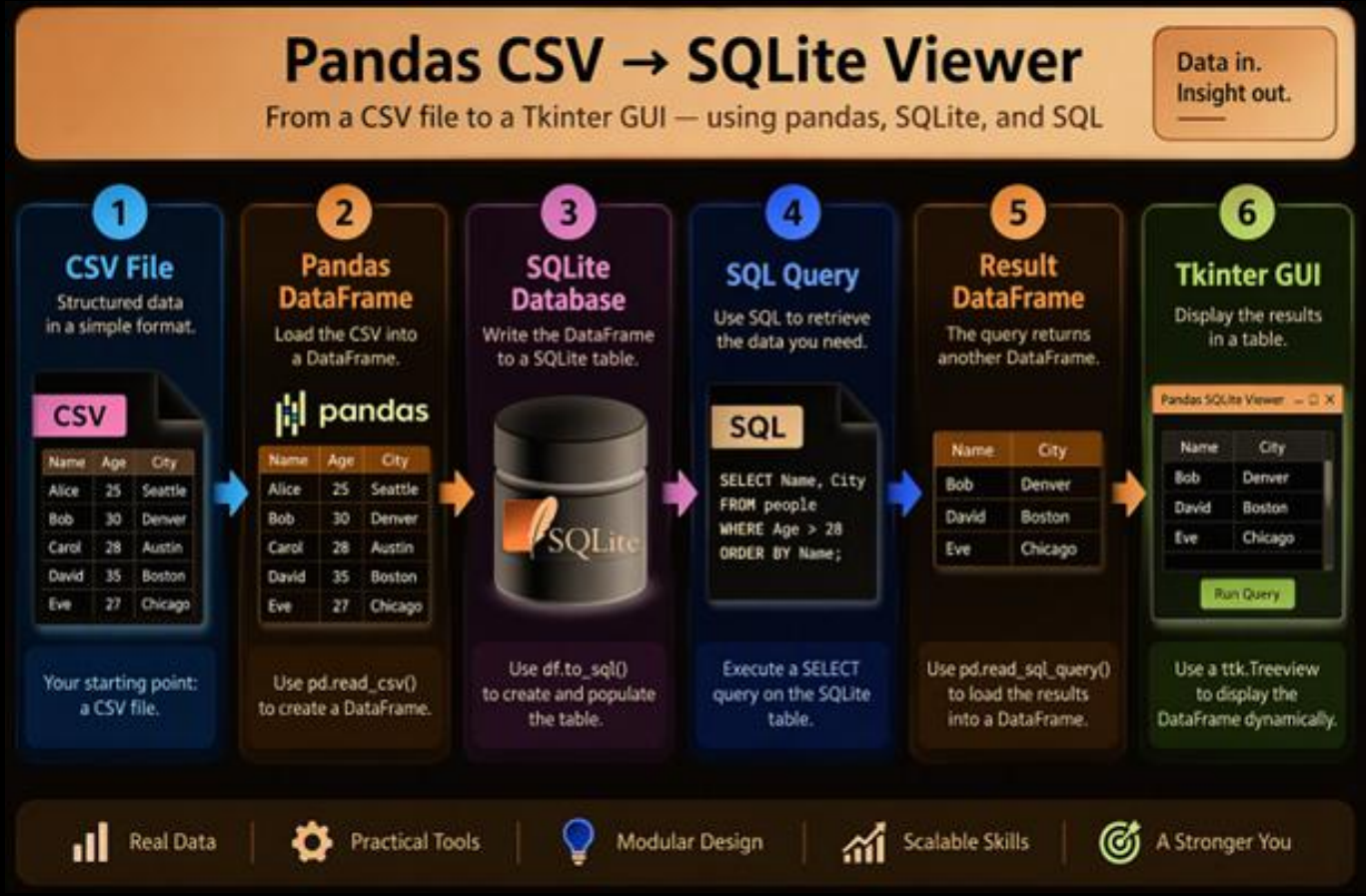
Pandas: CSV -> DataFrame -> SQLite -> Tkinter

A higher-level path from structured data to a GUI display



What changes with pandas?

The previous SQLite work stays useful, but pandas handles more of the movement of structured data.



- Before: CSV module -> dictionaries -> generated SQL -> SQLite
- Now: CSV -> DataFrame -> SQLite
- SQL is still used to retrieve the data we need
- The DataFrame becomes the bridge between CSV, SQLite, and the GUI

Read a CSV into a DataFrame

pd.read_csv() replaces the earlier CSV-to-dictionary loading step.

```
1 import pandas as pd
2
3 df = pd.read_csv("dogs.csv")
4 print(df)
```

A DataFrame organizes the CSV as rows and columns.

- df.columns gives the column names
- df.dtypes shows the data types pandas selected
- len(df) gives the number of rows

Work with columns and rows

The GUI will eventually need both pieces without hard-coded column names.

```
1 columns = list(df.columns)
2 print(columns)
3
4 for row in df.itertuples(index=False):
5     print(row)
```

Importance

- columns -> Treeview headings
- rows -> Treeview values
- the same display code can work with different query results

Create the SQLite connection

The connection is familiar. What happens next is different.




```
1 import sqlite3
2
3 conn = sqlite3.connect("dogs.db")
```

Write the DataFrame to SQLite

to_sql() creates and populates the table from the DataFrame.

```
df.to_sql(
    "dogs",
    conn,
    if_exists="replace",
    index=False
)
```

- No CREATE TABLE statement is written here
- No INSERT statements are written here
- if_exists="replace" replaces an existing table with the same name
- index=False keeps the DataFrame index from becoming a database column

SQL hasn't disappeared

Pandas handles table creation and insertion, but SQLite is still queried with SQL.

```
1 sql = """
2 SELECT *
3 FROM dogs
4 """
5
6 print(sql)
```

SQLite -> SQL SELECT -> DataFrame

The query can retrieve all rows or only the subset needed by the program.

Return the query as another DataFrame

read_sql_query() executes the SELECT and gives pandas the result set.

```
sql = """
SELECT name, breed, age
FROM dogs
WHERE age >= 7
"""

results = pd.read_sql_query(sql, conn)
```

```
results = pd.read_sql_query(sql, conn)
print(results)
```

CSV DataFrame -> SQLite -> filtered result DataFrame

Display a DataFrame with ttk.Treeview

Build headings and rows from the DataFrame instead of hard-coding the display.

```python
def display_dataframe(tree, df):
    tree.delete(*tree.get_children())
    columns = list(df.columns)
    tree["columns"] = columns
    tree["show"] = "headings"

    for column in columns:
        tree.heading(column, text=column)

    for row in df.itertuples(index=False, name=None):
        tree.insert("", "end", values=row)
```



Test the GUI display

The Treeview receives the DataFrame returned by the query.

```
root = tk.Tk()
root.title("Pandas SQL Results")

tree = ttk.Treeview(root)
tree.pack(fill="both",
expand=True, padx=10, pady=10)

display_dataframe(tree, results)

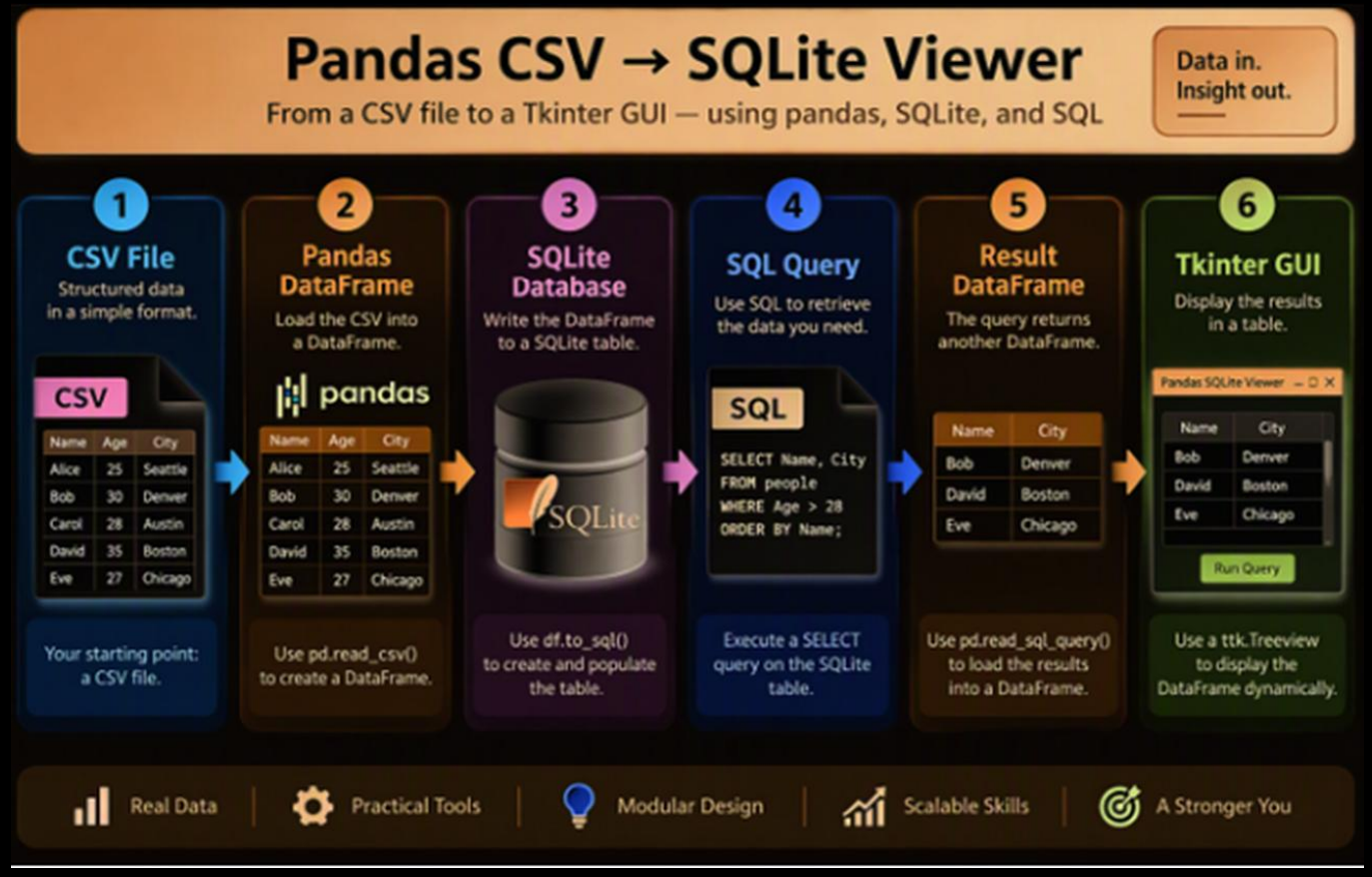
root.mainloop()
```

The GUI does not need to know the query columns ahead of time.

Those columns come from the DataFrame.

The complete pipeline

Each step now has a specific job.



CSV -> DataFrame -> SQLite -> SQL SELECT -> DataFrame -> Tkinter

Review

The assignment combines SQLite and Tkinter ideas with a new DataFrame-centered workflow.

- read a CSV file into a DataFrame
- examine DataFrame columns and data types
- work with DataFrame rows
- write a DataFrame to SQLite
- execute a SQL SELECT query
- receive query results as another DataFrame
- dynamically display a DataFrame in a Tkinter Treeview

Same data.
Higher-level tools.
Modular program design.